

2017年11月29日（水）
国際ロボット展RTM講習会



第 1 部 RTミドルウェア: OpenRTM-aist概要

国立研究開発法人産業技術総合研究所
ロボットイノベーション研究センター
ロボットソフトウェアプラットフォーム研究チーム長
安藤 慶昭

はじめに

- RTミドルウェアの概要
 - 基本概念
- RTコンポーネントの開発
 - RTCプログラミングについて
- RTMコミュニティー活動
 - サマーキャンプ、コンテスト、 etc.
- まとめ

RTミドルウェアとは？

RTとは?

- RT = Robot Technology cf. IT
 - ≠ Real-time
 - 単体のロボットだけでなく、さまざまなロボット技術に基づく機能要素をも含む (センサ、アクチュエータ, 制御スキーム、アルゴリズム、etc….)

産総研版RTミドルウェア

OpenRTM-aist

- RT-Middleware (RTM)
 - RT要素のインテグレーションのためのミドルウェア
- RT-Component (RTC)
 - RT-Middlewareにおけるソフトウェアの基本単位

ロボットミドルウェアについて

- ロボットシステム構築を効率化するための共通機能を提供する**基盤ソフトウェア**
 - － 「ロボットOS」と呼ばれることもある
 - － インターフェース・プロトコルの共通化、標準化
 - － 例として
 - モジュール化・コンポーネント化フレームワークを提供
 - モジュール間の通信をサポート
 - パラメータの設定、配置、起動、モジュールの複合化（結合）機能を提供
 - 抽象化により、OSや言語間連携・相互運用を実現
- 2000年ごろから開発が活発化
 - － 世界各国で様々なミドルウェアが開発・公開されている

従来のシステムでは…



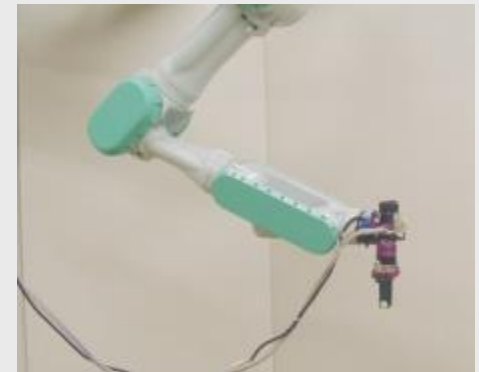
Joystick



Joystick
software



Robot Arm
Control software



Robot Arm

互換性のあるインターフェース同士は接続可能

従来のシステムでは…



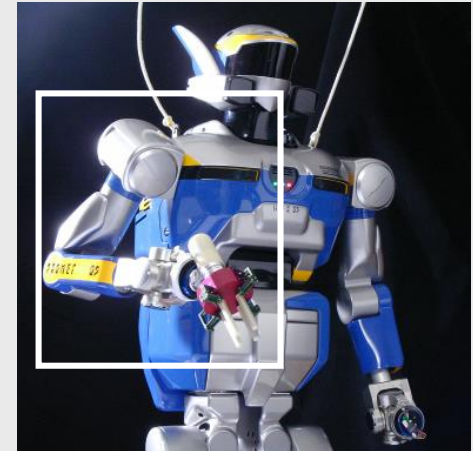
Joystick



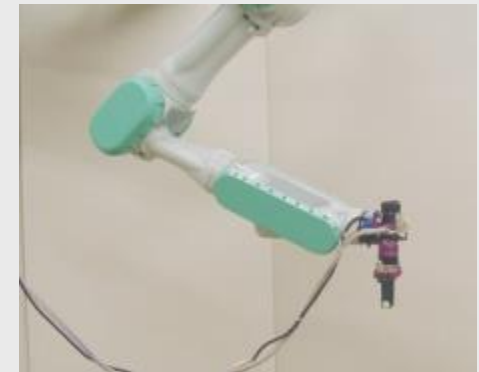
Joystick
software



Humanoid's Arm
Control software



Humanoid's Arm



Robot Arm

ロボットによって、インターフェースは色々
互換性が無ければつながらない

Robot Arm
Control software

RTミドルウェアでは…

RTミドルウェアは別々に作られたソフトウェアモジュール同士を繋ぐための共通インターフェースを提供する



Joystick

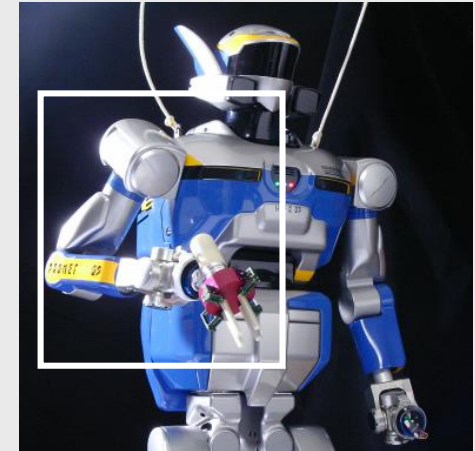


Joystick software

Arm A
Control software



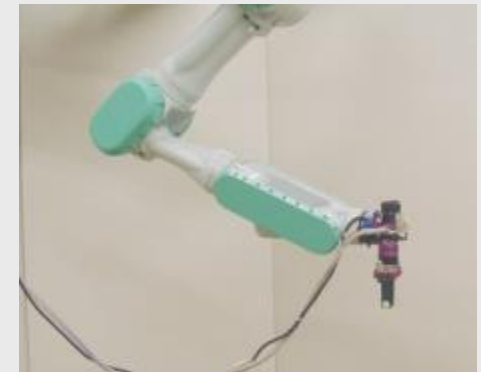
compatible
arm interfaces



Humanoid's Arm



Arm B
Control software



Robot Arm

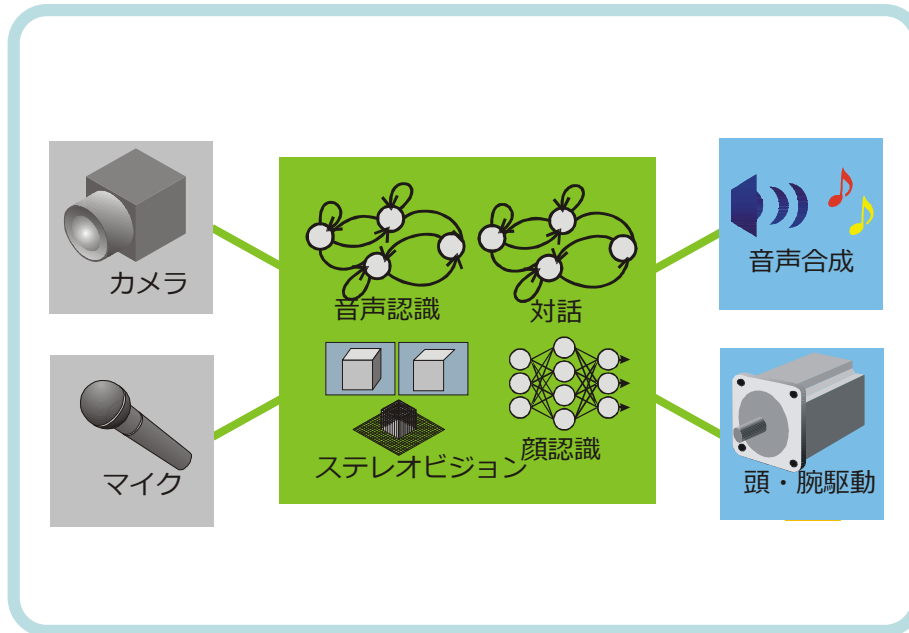
ソフトウェアの再利用性の向上
RTシステム構築が容易になる

ロボットソフトウェア開発の方向

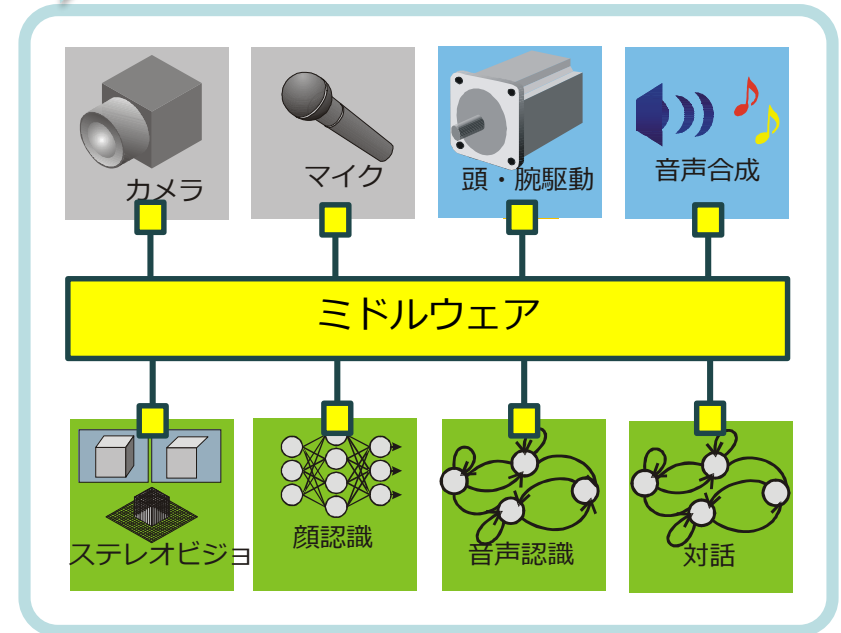
従来型開発



コンポーネント指向開発



- ✓ 様々な機能を融合的に設計
- ✓ 実行時の効率が高いが、柔軟性に欠ける
- ✓ システムが複雑化してくると開発が困難に



- ✓ 大規模複雑な機能の分割・統合
- ✓ 開発・保守効率化（機能の再利用等）
- ✓ システムの柔軟性向上

モジュール化のメリット

- 再利用性の向上
 - 同じコンポーネントをいろいろなシステムに使いまわせる
- 選択肢の多様化
 - 同じ機能を持つ複数のモジュールを試すことができる
- 柔軟性の向上
 - モジュール接続構成かえるだけで様々なシステムを構築できる
- 信頼性の向上
 - モジュール単位でテスト可能なため信頼性が向上する
- 堅牢性の向上
 - システムがモジュールで分割されているので、一つの問題が全体に波及しにくい

RTコンポーネント化のメリット

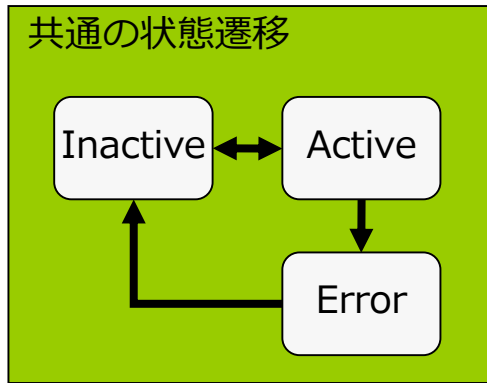
モジュール化のメリットに加えて

- ソフトウェアパターンを提供
 - ロボットに特有のソフトウェアパターンを提供することで、体系的なシステム構築が可能
- フレームワークの提供
 - フレームワークが提供されているので、コアのロジックに集中できる
- 分散ミドルウェア
 - ロボット体内LANやネットワークロボットなど、分散システムを容易に構築可能

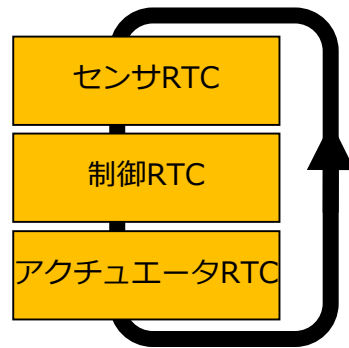
RTコンポーネントの主な機能

アクティビティ・実行コンテキスト

共通の状態遷移



複合実行

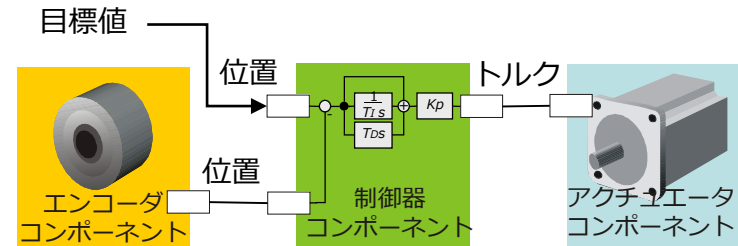


ライフサイクルの管理・コアロジックの実行

データポート

- データ指向ポート
- 連続的なデータの送受信
- 動的な接続・切断

サーボの例



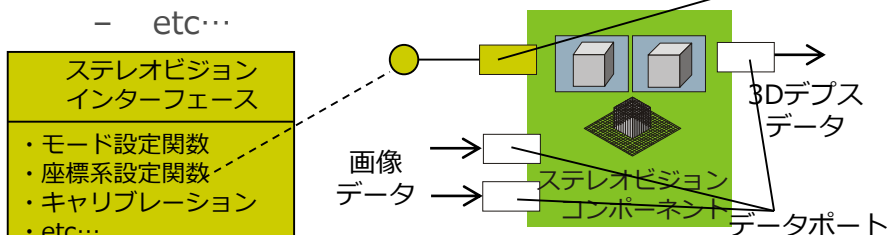
データ指向通信機能

サービスポート

- 定義可能なインターフェースを持つ
- 内部の詳細な機能にアクセス
 - パラメータ取得・設定
 - モード切替
 - etc...

ステレオビジョンの例

サービスポート



サービス指向相互作用機能

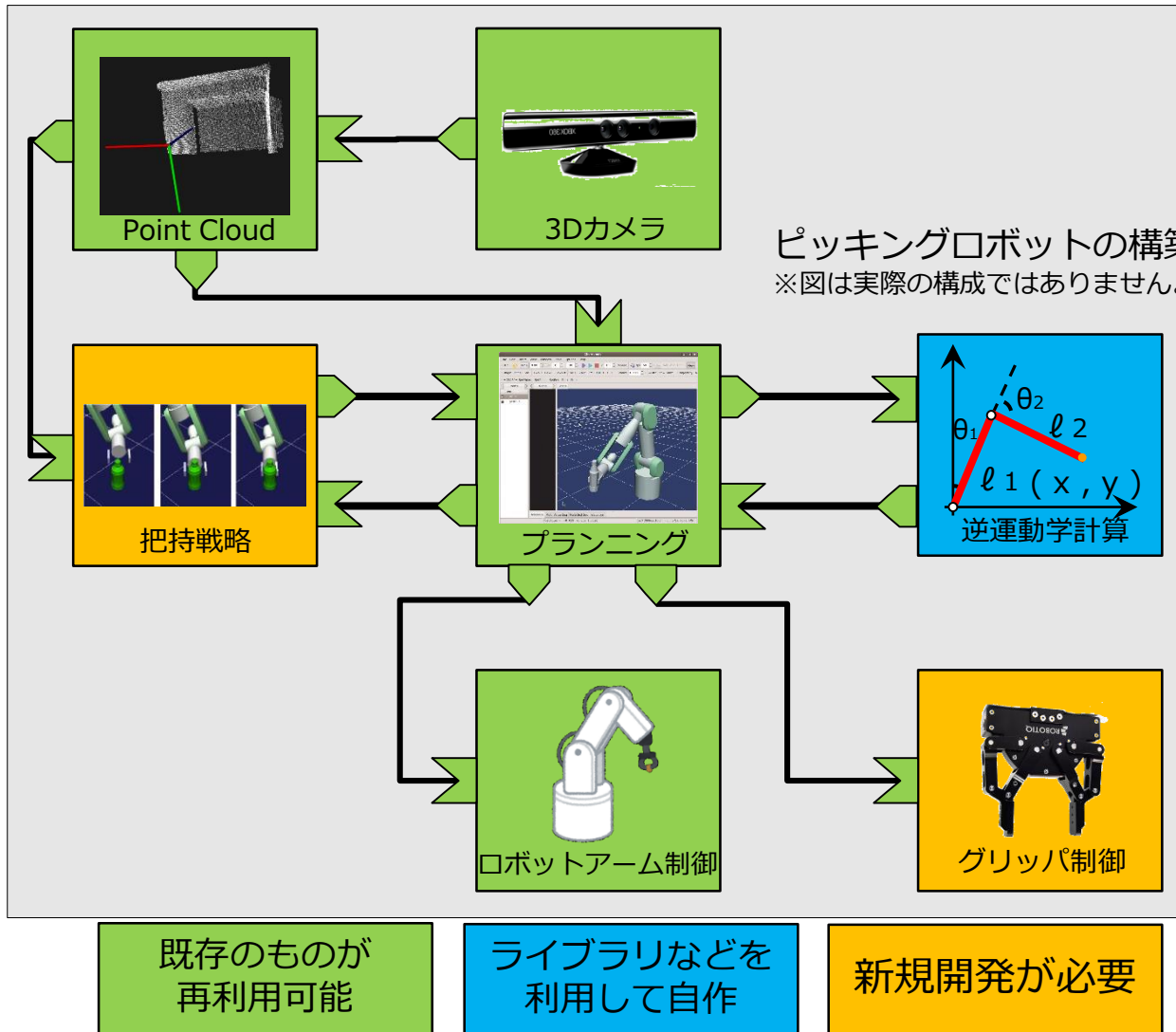
コンフィギュレーション

- パラメータを保持する仕組み
- いくつかのセットを保持可能
- 実行時に動的に変更可能

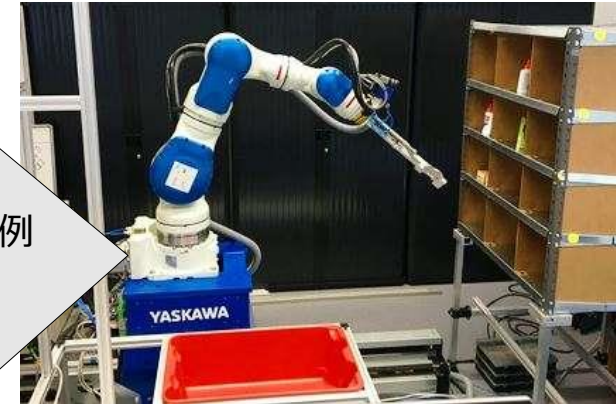
複数のセットを
動作時に
切り替えて
使用可能

セット名	名前	値			
	値				
セット名	名前	値			
	値				

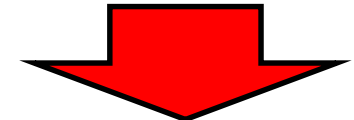
ミドルウェアを利用した開発の利点



ピッキングロボットの構築例
※図は実際の構成ではありません。



ミドルウェアを利用すると、**既存のモジュール**が利用できる

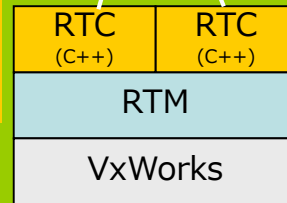


開発するときに**新規に作らなければならない部分**は少なくて済む

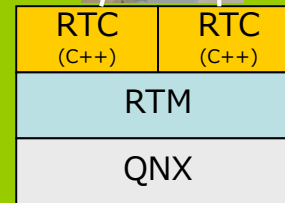
RTミドルウェアによる分散システム

RTMにより、
ネットワーク上に
分散するRTCを
OS・言語の壁を
越えて接続する
ことができる。

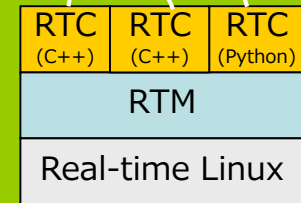
ロボットA



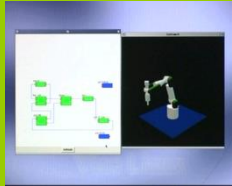
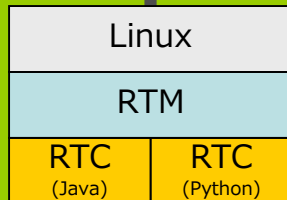
ロボットB



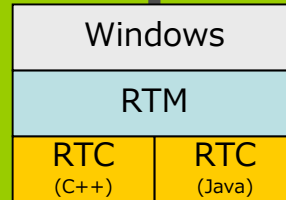
ロボットC



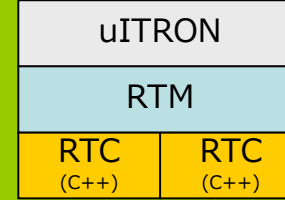
ネットワーク



アプリケーション



操作デバイス



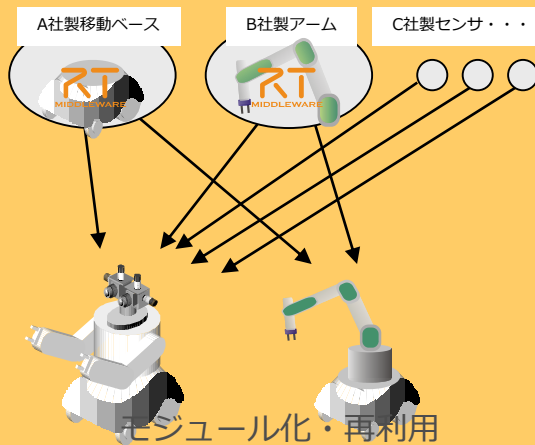
センサ

RTC同士の接続
は、プログラム
実行中に動的に
行うことができる。

RTミドルウェアの目的

モジュール化による問題解決

コストの問題



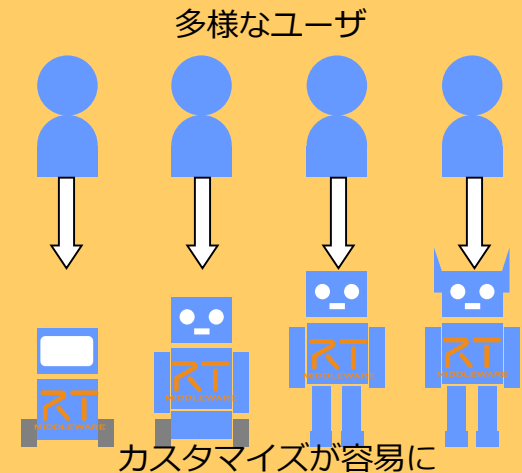
ロボットの低コスト化

技術の問題



最新技術を利用可能

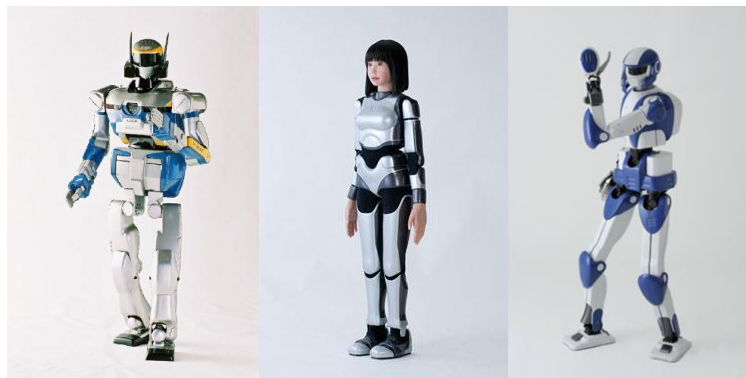
ニーズの問題



多様なニーズに対応

ロボットシステムインテグレーションによるイノベーション

実用例・製品化例



HRPシリーズ: 川田工業、AIST

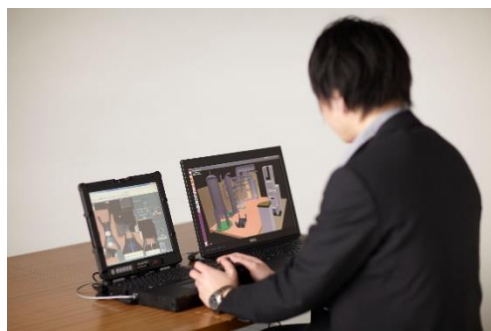


S-ONE : SCHAFT



DAQ-Middleware: KEK/J-PARC

KEK: High Energy Accelerator Research Organization
J-PARC: Japan Proton Accelerator Research Complex



災害対応ロボット操縦シミュレータ:
NEDO/千葉工大



HIRO, NEXTAGE open: Kawada Robotics



RAPUDA : Life Robotics



ビュートローバーRTC/RTC-BT(VSTONE)



OROCHI (アルティ)



新日本電工他: Mobile SEM

RTミドルウェアは国際標準

OMG国際標準

- 標準化組織で手続きに沿って策定
- 1組織では勝手に改変できない安心感
 - 多くの互換実装ができてつある
 - 競争と相互運用性が促進される

RTミドルウェア互換実装は10種類以上

名称	ベンダ	特徴	互換性
OpenRTM-aist	産総研	NEDO PJで開発。参照実装。	---
HRTM	ホンダ	アシモはHRTMへ移行中	◎
OpenRTM.NET	セック	.NET(C#, VB, C++/CLI, F#, etc..)	◎
RTM on Android	セック	Android版RTミドルウェア	◎
RTC-Lite	産総研	PIC, dsPIC上の実装	○
Mini/MicorRTC	SEC	NEDOオープンイノベーションPJで開発	○
RTMSafety	SEC/AIST	NEDO知能化PJで開発・機能安全認証取得	○
RTC CANOpen	SIT, CiA	CAN業界RTM標準	○
PALRO	富士ソフト	小型ヒューマノイドのためのC++ PSM 実装	×
OPRoS	ETRI	韓国国家プロジェクトでの実装	×
GostaiRTC	GOSTAI, THALES	ロボット言語上で動作するC++ PSM 実装	×

特定のベンダが撤退しても
ユーザは使い続けることが可能



Robotic Technology Component (RTC)

Version 1.1

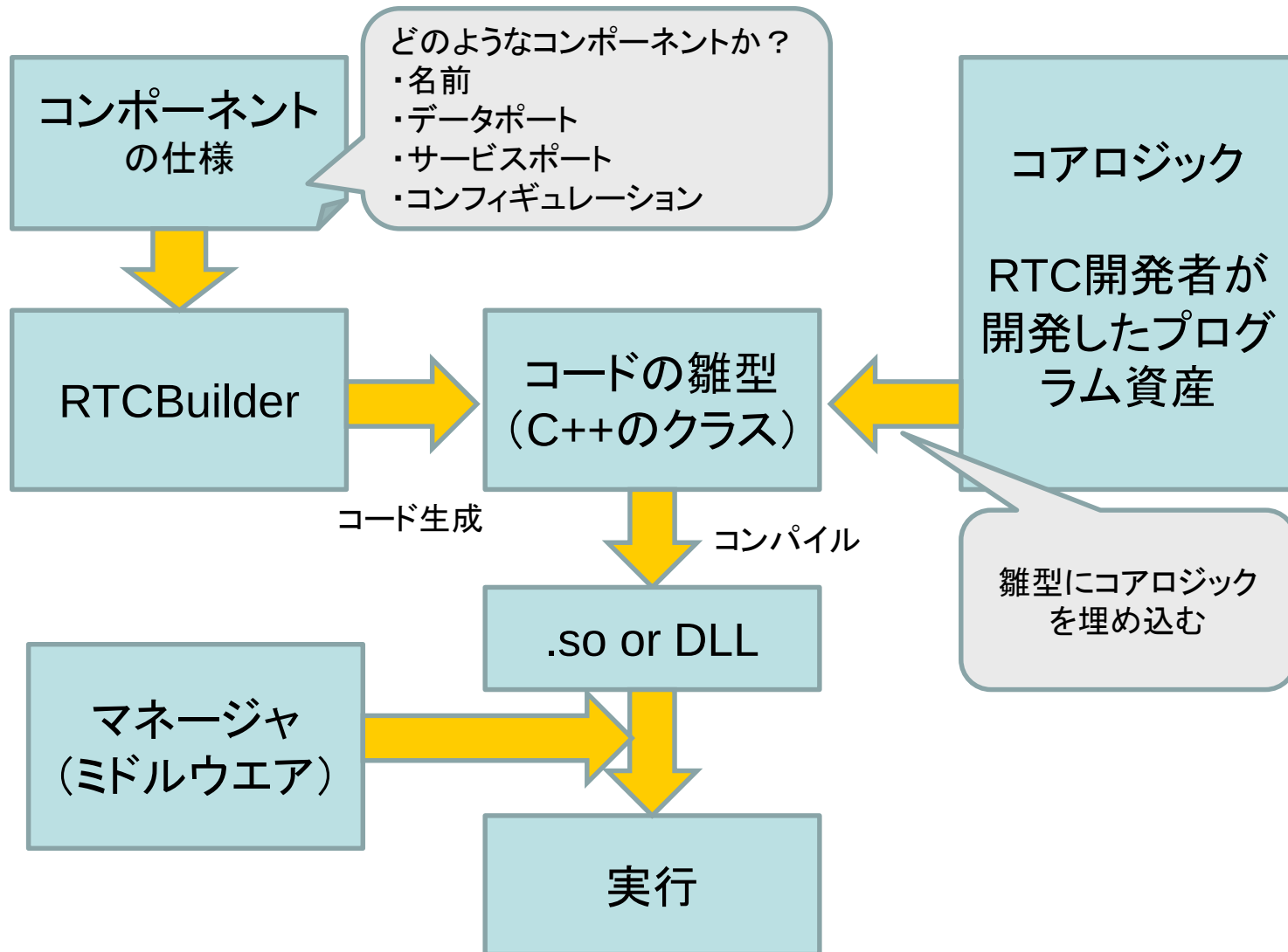
Normative reference: <http://www.omg.org/spec/RTC/1.1>
Machine consumable files: <http://www.omg.org/spec/RTC/20111205/>
Normative:
<http://www.omg.org/spec/RTC/20111205/rtc.xml>
<http://www.omg.org/spec/RTC/20111205/rtc.h>
<http://www.omg.org/spec/RTC/20111205/rtc.idl>
Non-normative:
<http://www.omg.org/spec/RTC/20111205/rtc.eap>

標準化履歴

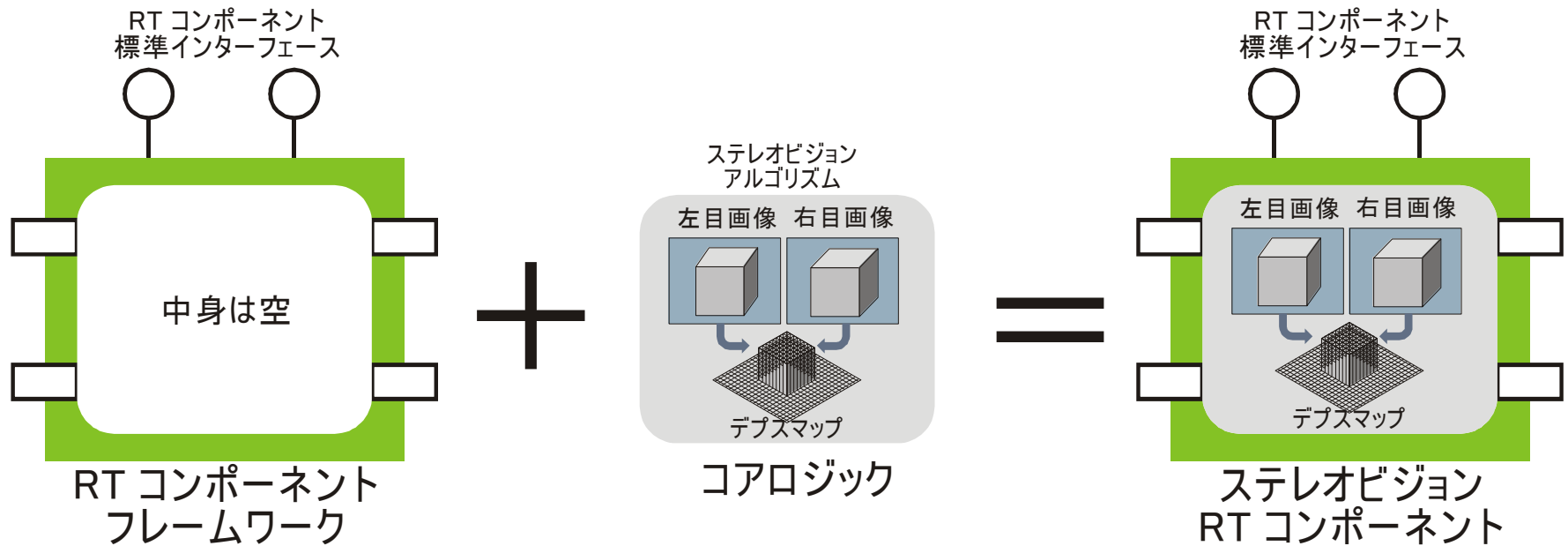
- 2005年9月
Request for Proposal 発行(標準化開始)
- 2006年9月
OMGで承認、事実上の国際標準獲得
- 2008年4月
OMG RTC標準仕様 ver.1.0公式リリース
- 2012年9月
ver. 1.1改定
- 2015年9月
FSM4RTC(FSM型RTCとデータポート標準) 採択

RTコンポーネントの開発

OpenRTMを使った開発の流れ

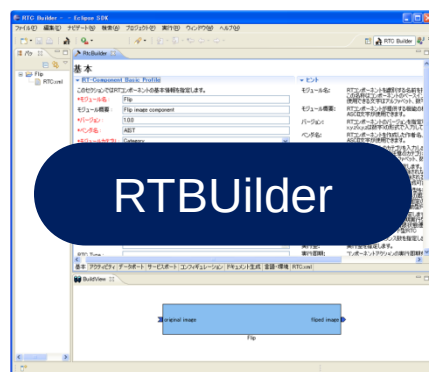


フレームワークとコアロジック

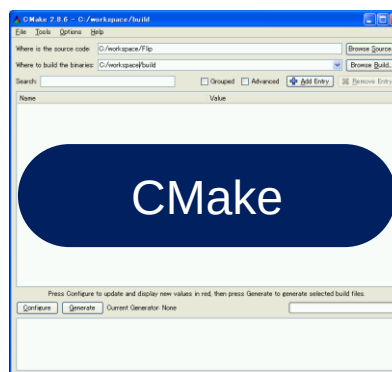
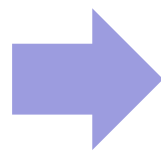


RTCフレームワーク+コアロジック=RTコンポーネント

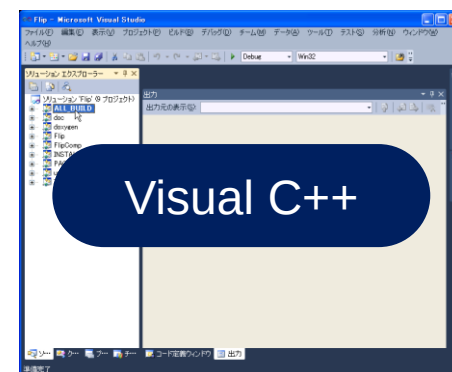
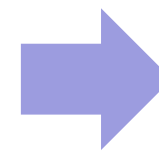
コンポーネントの作成 (Windowsの場合)



コンポーネントの
仕様の入力



VCのプロジェクト
ファイルの生成



実装および
VCでコンパイル
実行ファイルの生成



テンプレートコード
の生成



コード例

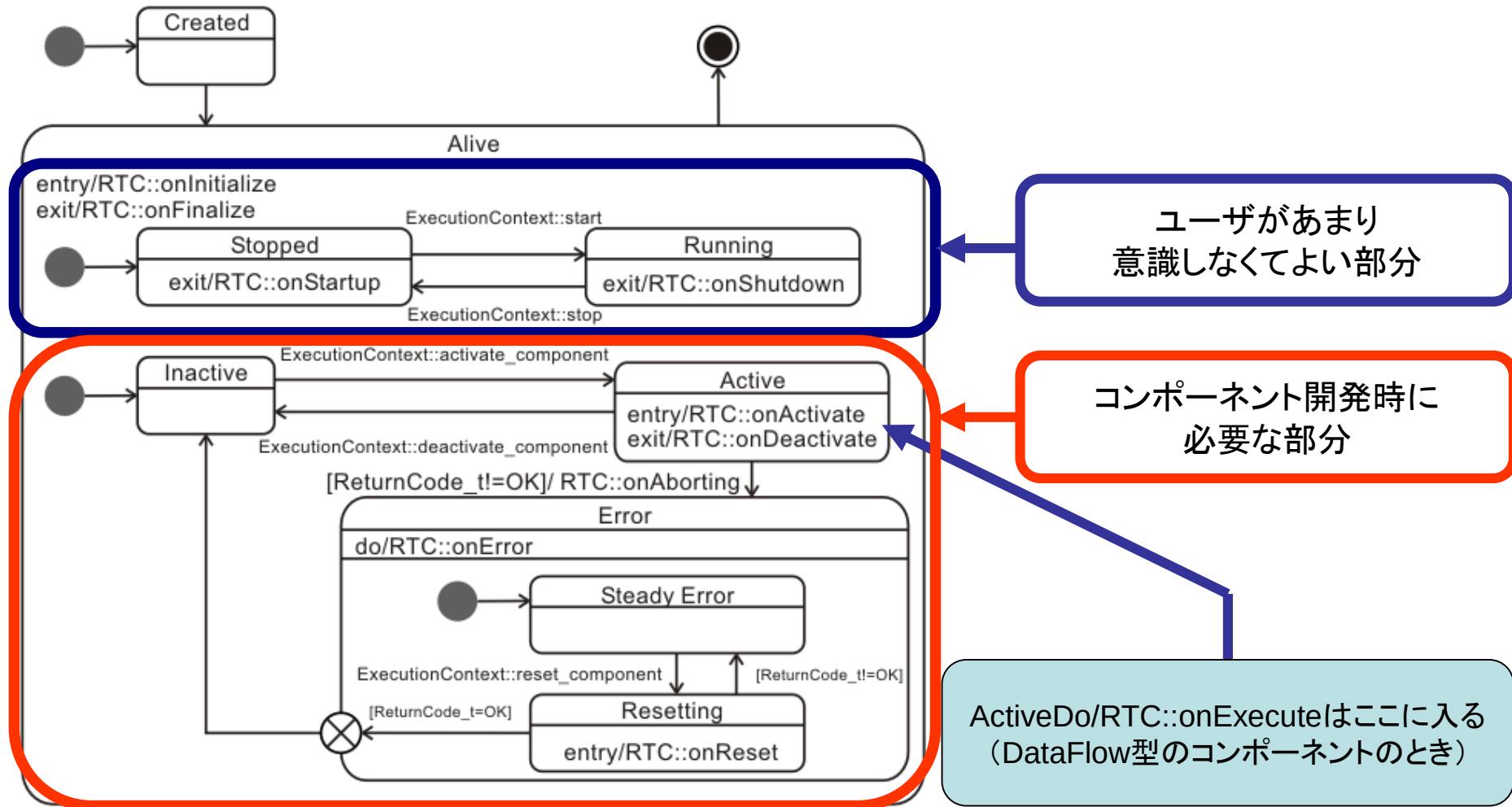
- 生成されたクラスのメンバー関数に必要な処理を記述
- 主要な関数
 - onExecute (周期実行)
- 処理
 - InPortから読む
 - OutPortへ書く
 - サービスを呼ぶ
 - コンフィギュレーションを読む

```
class MyComponent
: public DataflowComponentBase
{
public:
    // 初期化時に実行したい処理
    virtual ReturnCode_t onInitialize()
    {
        if (mylogic.init())
            return RTC::RTC_OK;
        return RTC::RTC_ERROR;
    }

    // 周期的に実行したい処理
    virtual ReturnCode_t onExecute(RTC::UniqueId ec_id)
    {
        if (mylogic.do_something())
            return RTC::RTC_OK;
        return RTC::RTC_ERROR;
    }

private:
    MyLogic mylogic;
    // ポート等の宣言
    //   :
};
```

コンポーネント内の状態遷移



コールバック関数

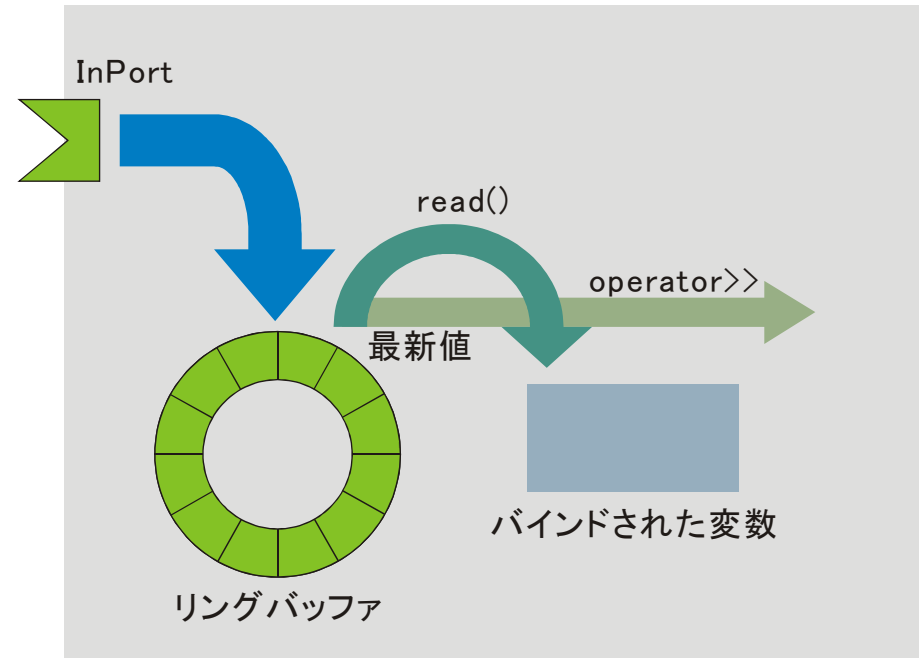
RTCの作成＝コールバック関数に処理を埋め込む

コールバック関数	処理
onInitialize	初期化処理
onActivated	アクティブ化されるとき1度だけ呼ばれる
onExecute	アクティブ状態時に周期的に呼ばれる
onDeactivated	非アクティブ化されるとき1度だけ呼ばれる
onAborting	ERROR状態に入る前に1度だけ呼ばれる
onReset	resetされる時に1度だけ呼ばれる
onError	ERROR状態のときに周期的に呼ばれる
onFinalize	終了時に1度だけ呼ばれる
onStateUpdate	onExecuteの後毎回呼ばれる
onRateChanged	ExecutionContextのrateが変更されたとき1度だけ呼ばれる
onStartup	ExecutionContextが実行を開始するとき1度だけ呼ばれる
onShutdown	ExecutionContextが実行を停止するとき1度だけ呼ばれる

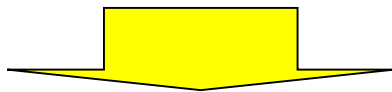
とりあえずは
この5つの関数
を押さえて
おけばOK

InPort

- InPortのテンプレート第2引数: バッファ
 - ユーザ定義のバッファが利用可能
- InPortのメソッド
 - read(): InPort バッファからバインドされた変数へ最新値を読み込む
 - >>: ある変数へ最新値を読み込む

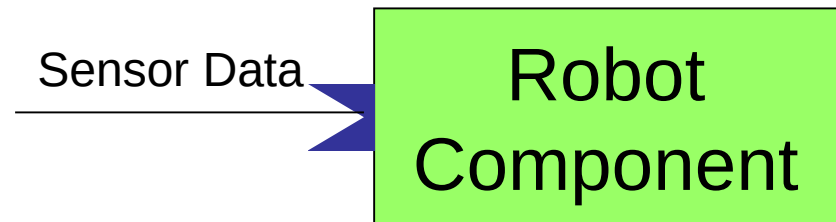


基本的にOutPortと対になる



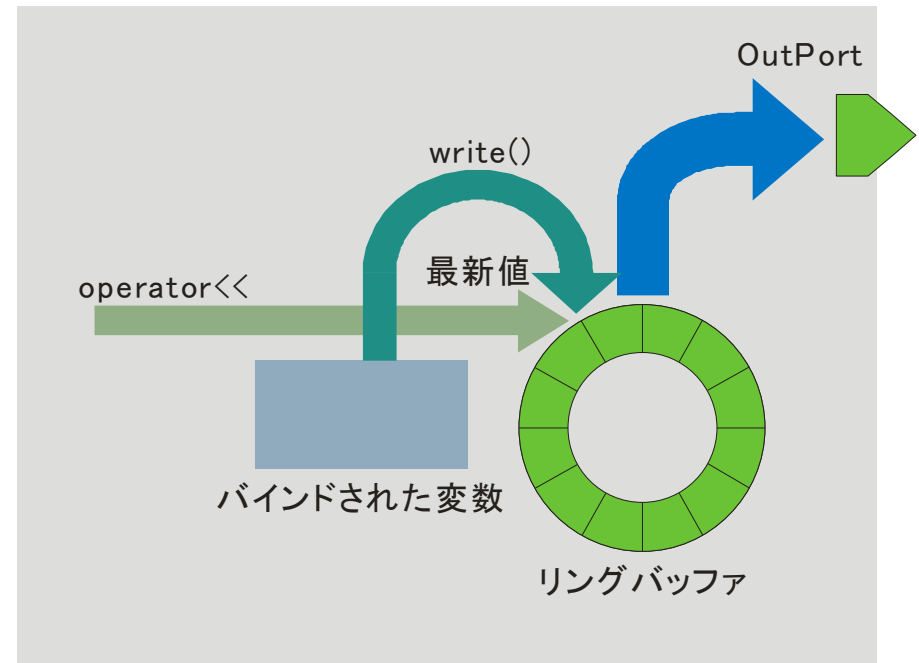
データポートの型を
同じにする必要あり

例



OutPort

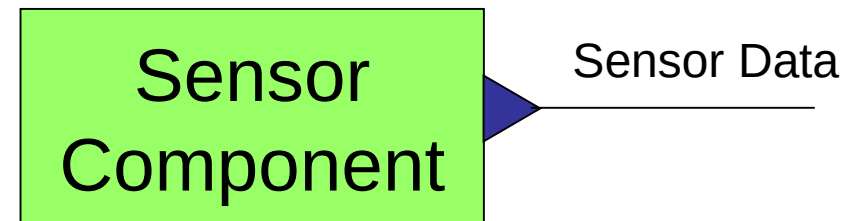
- OutPortのテンプレート第2引数:
バッファ
 - ユーザ定義のバッファが利用可能
- OutPortのメソッド
 - write(): OutPort バッファへ
バインドされた変数の最新値
として書き込む
 - >>: ある変数の内容を最新
値としてリングバッファに書き
込む



基本的にInPortと対になる

データポートの型を
同じにする必要あり

例



データ変数

```
struct TimedShort
{
    Time tm;
    short data;
};
```

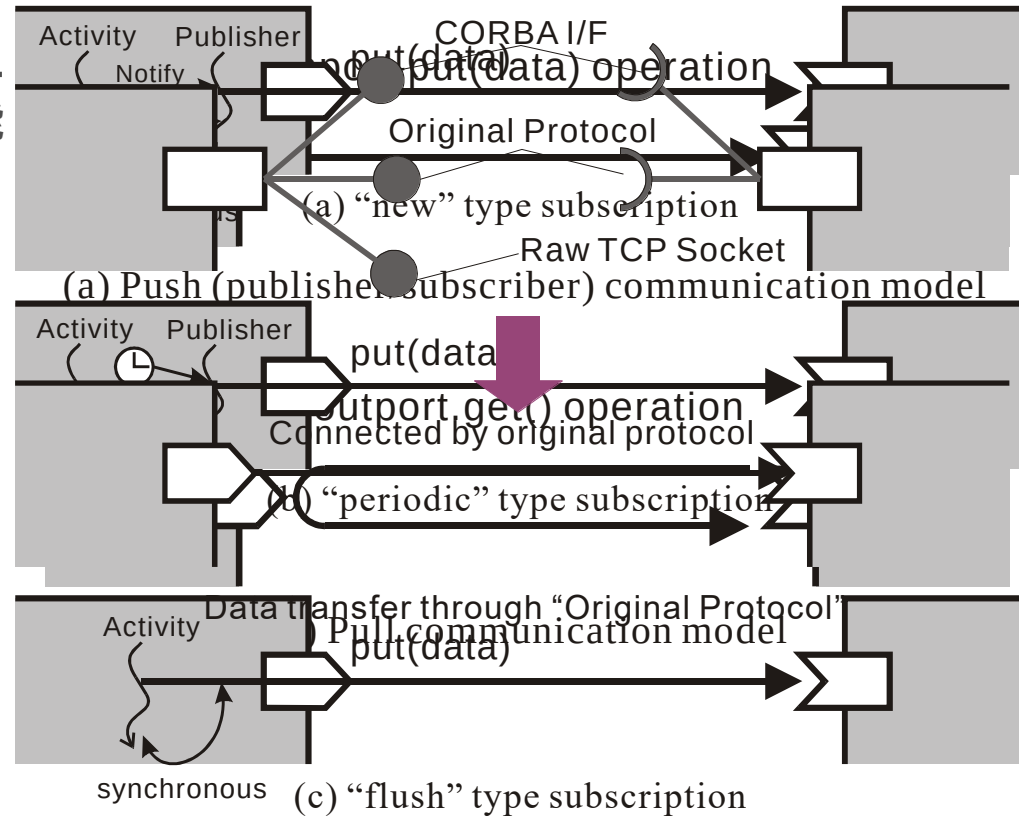
- 基本型
 - tm: 時刻
 - data: データそのもの

```
struct TimedShortSeq
{
    Time tm;
    sequence<short> data;
};
```

- シーケンス型
 - data[i]: 添え字によるアクセス
 - data.length(i): 長さiを確保
 - data.length(): 長さを取得
- データを入れるときにはあらかじめ長さをセットしなければならない。
- CORBAのシーケンス型そのもの
- 今後変更される可能性あり

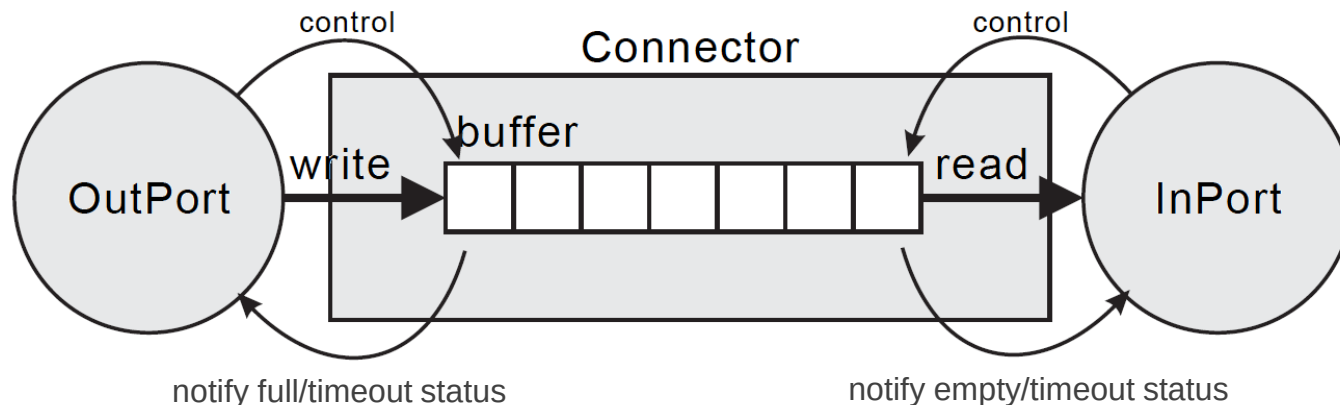
データポート

- データ指向 (Data Centric) な
ストリームポート
 - 型: long, double × 6, etc...
 - ユーザが任意に定義可能
 - 出力: OutPort
 - 入力: InPort
- 接続制御 (接続時に選択可能)
 - Interface type
 - CORBA, TCP socket, other protocol, etc...
 - Data flow type
 - push/pull
 - Subscription type
 - Flush, New, Periodic



データポートモデル

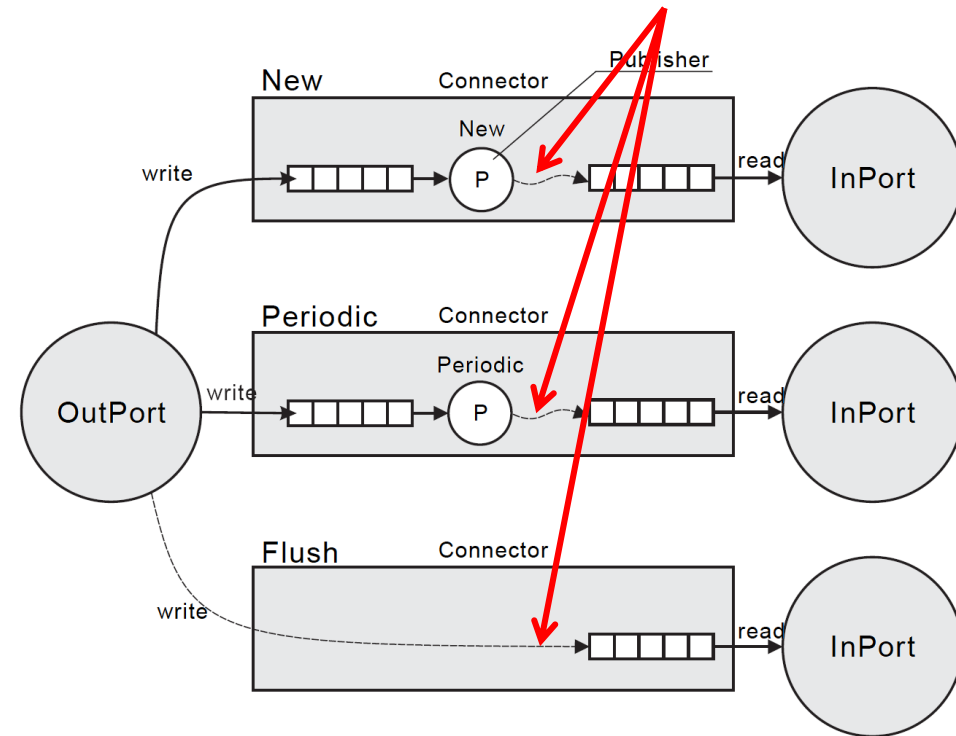
- Connector:
 - バッファと通信路を抽象化したオブジェクト。OutPortからデータを受け取りバッファに書き込む。InPortからの要求に従いバッファからデータを取り出す。
 - OutPortに対してバッファフル・タイムアウト等のステータスを伝える。
 - InPortに対してバッファエンプティ・タイムアウト等のステータスを伝える。
- OutPort:
 - アクティビティからの要求によってデータをコネクタに書き込むオブジェクト
- InPort:
 - アクティビティからの要求によってデータをコネクタから読み出すオブジェクト



Push型データポートモデル

- Connector
 - 実際には間に通信が入る可能性がある
- 3つの送信モデル
 - “new”, “periodic”, “flush”
 - パブリッシャによる実現
- バッファ、パブリッシャ、通信インターフェースの3つをConnectorに内包

ネットワーク等による通信



バッファインターフェース

- バッファ操作のためのインターフェースを定義
 - より詳細な操作を提供
 - バッファフル/エンプティを検出可能に
 - タイムアウトを検出可能に

BufferBase <T>

```
advanceRptr(long n) : ReturnCode  
advanceWptr(long n) : ReturnCode  
empty() : bool  
full() : bool  
get(T& data) : ReturnCode  
put(T& data) : ReturnCode  
read(T& data, int sec, int nsec) : ReturnCode  
write(T& data, int sec, int nsec) : ReturnCode
```

リターンコード

- BUFFER OK 正常終了
- BUFFER ERROR エラー終了
- BUFFER FULL バッファフル状態
- BUFFER EMPTY バッファ空状態
- NOT SUPPORTED サポート外の要求
- TIMEOUT タイムアウト
- PRECONDITION NOT MET 事前状態を満たさない

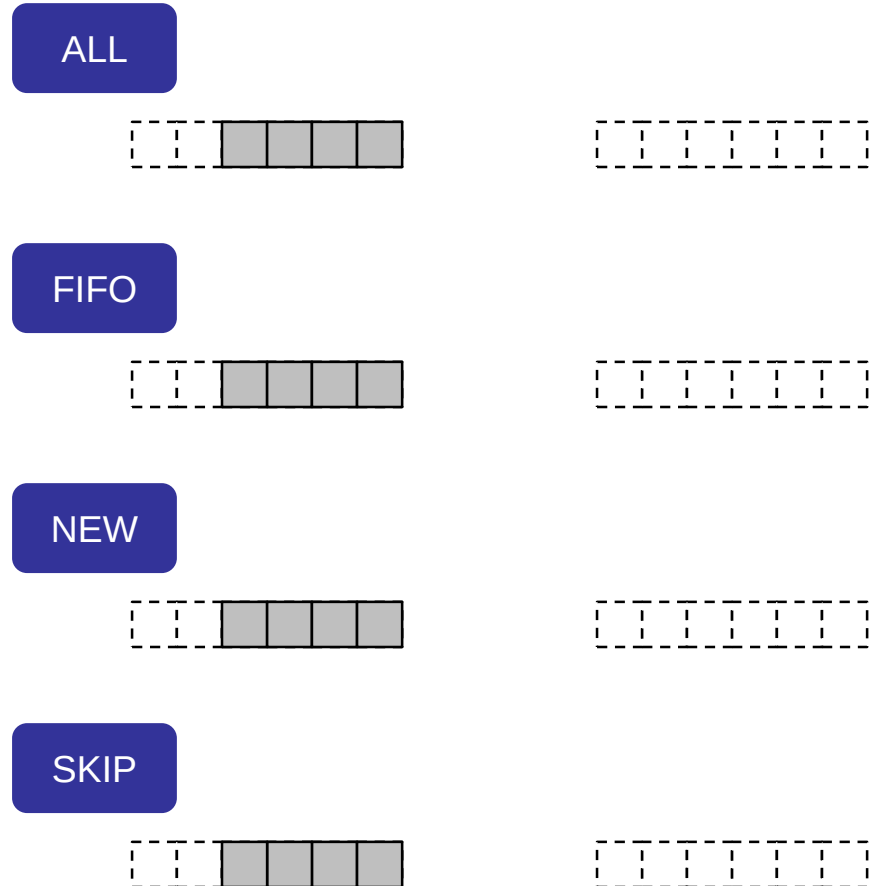
Pushポリシー

- バッファ残留データ
– 送り方のポリシー

ポリシー	送り方
ALL	全部送信
FIFO	先入れ後だしで1個ずつ送信
NEW	最新値のみ送信
SKIP	n個おきに間引いて送信

- データ生成・消費速度を考慮して設定する必要がある。

予稿にはNEWの説明にLIFOと記述していましたが正確にはLIFOではなく最新値のみの送信です。LIFO形式のポリシーを導入するかどうかは検討中です。ご意見ください。



rtc.confについて

RT Component起動時の登録先NamingServiceや、登録情報などについて記述するファイル

記述例:

corba.nameservers: localhost:9876

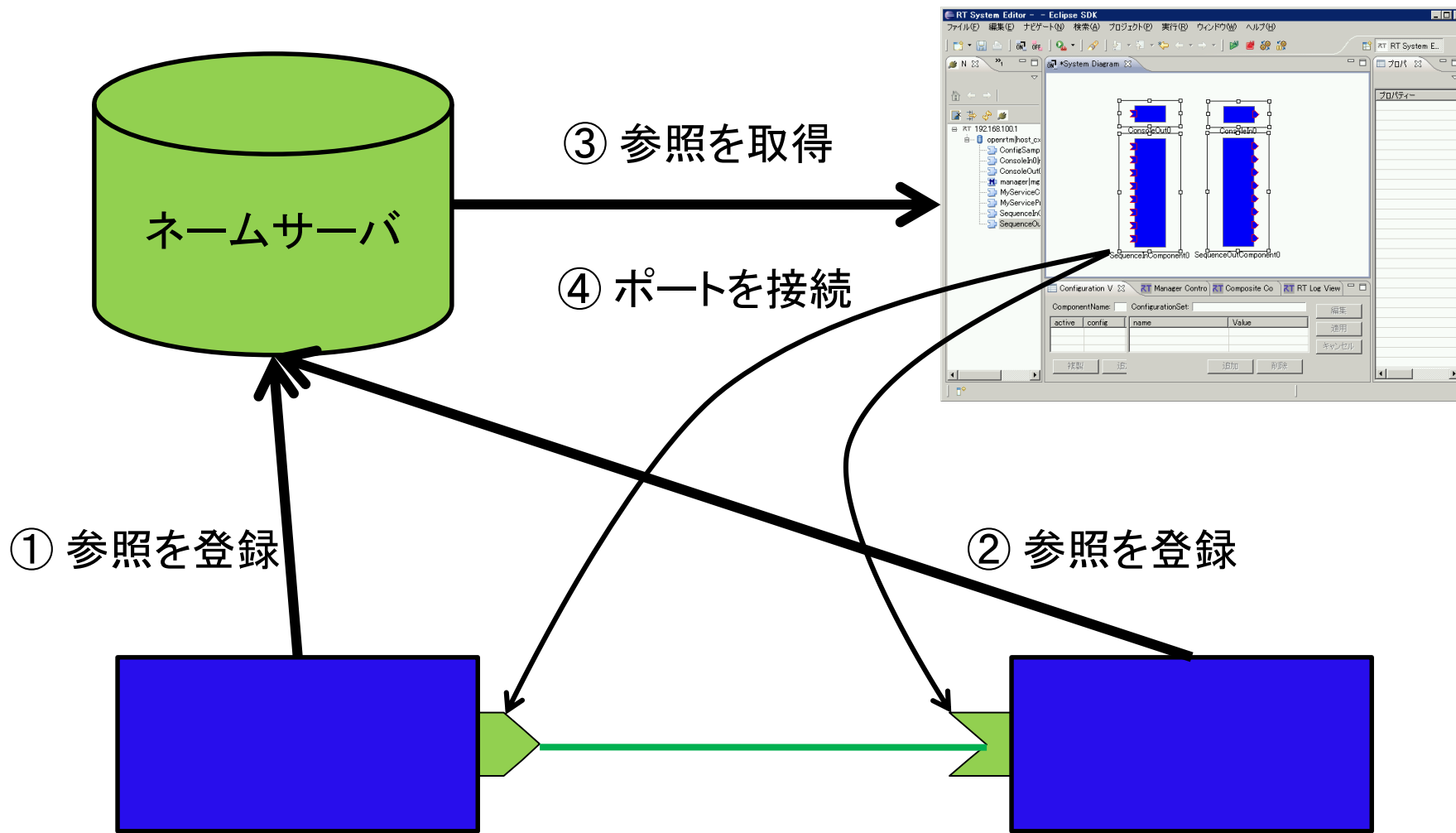
naming.formats: SimpleComponent/%n.rtc

(詳細な記述方法は etc/rtc.conf.sample を参照)

以下のようにすると、コンポーネント起動時に読み込まれる

```
./ConsoleInComp -f rtc.conf
```

動作シーケンス



ネーミングサービス設定

corba.nameservers	host_name:port_numberで指定、デフォルトポートは2809(omniORBのデフォルト)、複数指定可能
naming.formats	%h.host_cxt/%n.rtc → host.host_cxt/MyComp.rtc 複数指定可能、0.2.0互換にしたければ、 %h.host_cxt/%M.mgr_cxt/%c.cat_cxt/%m.mod_cxt/%n.rtc
naming.update.enable	“YES” or “NO”: ネーミングサービスへの登録の自動アップデート。コンポーネント起動後にネームサービスが起動したときに、再度名前を登録する。
naming.update.interval	アップデートの周期[s]。デフォルトは10秒。
timer.enable	“YES” or “NO”: マネージャタイマ有効・無効。 naming.updateを使用するには有効でなければならない
timer.tick	タイマの分解能[s]。デフォルトは100ms。



必須の項目



必須でないOption設定

ログ設定

logger.enable	“YES” or “NO”: ログ出力を有効・無効
logger.file_name	ログファイル名。 %h: ホスト名、%M: マネージャ名、%p: プロセスID 使用可
logger.date_format	日付フォーマット。strftime(3)の表記法に準拠。 デフォルト: %b %d %H:%M:%S → Apr 24 01:02:04
logger.log_level	ログレベル: SILENT, ERROR, WARN, NORMAL, INFO, DEBUG, TRACE, VERBOSE, PARANOID SILENT: 何も出力しない PARANOID: 全て出力する ※以前はRTC内で使えましたが、現在はまだ使えません 。

 必須の項目 必須でないOption設定

その他

corba.endpoints	IP_Addr:Port で指定 : NICが複数あるとき、ORBをどちらでlistenさせるかを指定。Portを指定しない場合でも”:”が必要。 例 “corba.endpoints: 192.168.0.12:” NICが2つある場合必ず指定。 (指定しなくても偶然正常に動作することもあるが念のため。)
corba.args	CORBAに対する引数。詳細はomniORBのマニュアル参照。
[カテゴリ名]. [コンポーネント名]. config_file または [カテゴリ名]. [インスタンス名]. config_file	コンポーネントの設定ファイル •カテゴリ名 : manipulator, •コンポーネント名 : myarm, •インスタンス名 myarm0,1,2,... の場合 manipulator.myarm.config_file: arm.conf manipulator.myarm0.config.file: arm0.conf のように指定可能

使いたいNICに割り当てられているIPアドレス



必須の項目



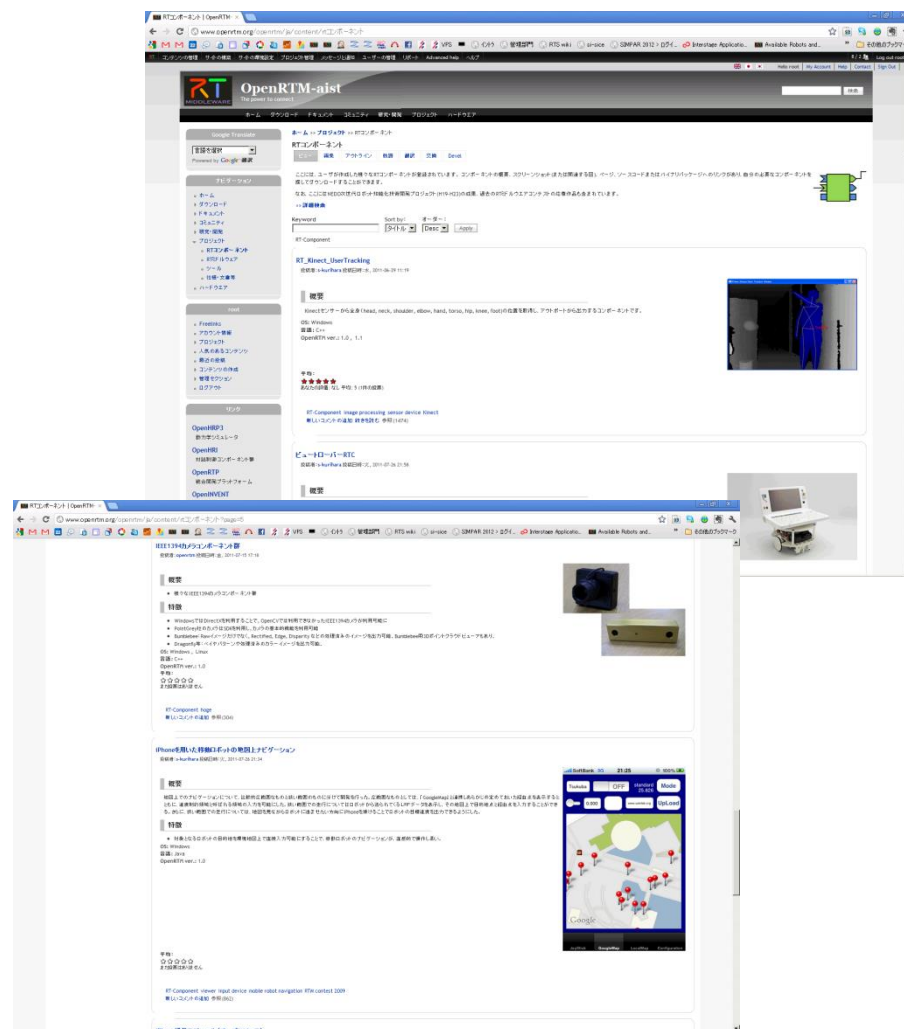
必須でないOption設定

RTミドルウェアと コミュニティ

プロジェクトページ

- ユーザが自分の作品を登録
- 他のユーザの作ったRTCを探すことができる

タイプ	登録数
RTコンポーネント群	287
RTミドルウェア	14
ツール	19
仕様・文書	4
ハードウェア	28



サマーキャンプ

- 毎年夏に1週間開催
- 今年：7月31日～8月4日
- 募集人数：20名
- 場所：産総研つくばセンター
- 座学と実習を1週間行い、最後にそれぞれが成果を発表
- 産総研内のさくら館に宿泊しながら夜通し？コーディングを行う！



RTミドルウェアコンテスト

- SICE SI（計測自動制御学会 システムインテグレーション部門講演会）のセッションとして開催
 - 各種奨励賞・審査基準開示:5月頃
 - エントリー〆切：8月21日(SI2017締切)
 - 講演原稿〆切：9月25日
 - ソフトウェア登録：10月ごろ
 - オンライン審査：11月下旬～
 - 発表・授賞式：12月ごろ
- 2016年度実績
 - 応募数：13件
 - 計測自動制御学会学会RTミドルウェア賞（副賞10万円）
 - 奨励賞（賞品協賛）：2件
 - 奨励賞（団体協賛）：8件
 - 奨励賞（個人協賛）：5件
- 詳細はWebページ：openrtm.org
 - コミュニティー→イベント をご覧ください



提言

- 自前主義はやめよう！！
 - 書きたてのコードより、いろいろな人に何万回も実行されたコードのほうが動くコードである！！
 - 自分にとって本質的でない部分は任せて、本当にやりたい部分・やるべき部分のコードを書こう！！
 - 誰かがリリースしたプログラムは一度は動いたことがあるプログラムである！！
 - 人のコードを読むのが面倒だからと捨ててしまうのはもったいない！！
- オープンソースにコミットしよう！！
 - 臆せずMLやフォーラムで質問しよう！！
 - どんなに初歩的な質問でも他の人にとっては価値ある情報である。
 - 要望を積極的にあげよう！！
 - できればデバッグしてパッチを送ろう！

まとめ

- RTミドルウェアの概要
 - 基本概念
- RTコンポーネントの開発
 - RTCプログラミングについて
- RTMコミュニティ活動
 - サマーキャンプ、コンテスト、 etc.

本日の実習の流れ

第2部: RTコンポーネントの作成入門

- 担当: 宮本信彦 氏 (産総研) 他
- 内容: RTコンポーネント設計ツールRTCBuilderとRTシステム構築ツールRTSystemEditorの利用方法を解説するとともに、移動ロボットシミュレータを用いた実習によりRTコンポーネントの開発手順、動作確認手順を学習します。また、作成したコンポーネントを利用し、移動ロボット実機(RaspberryPiマウス)を制御する方法についても学習します。

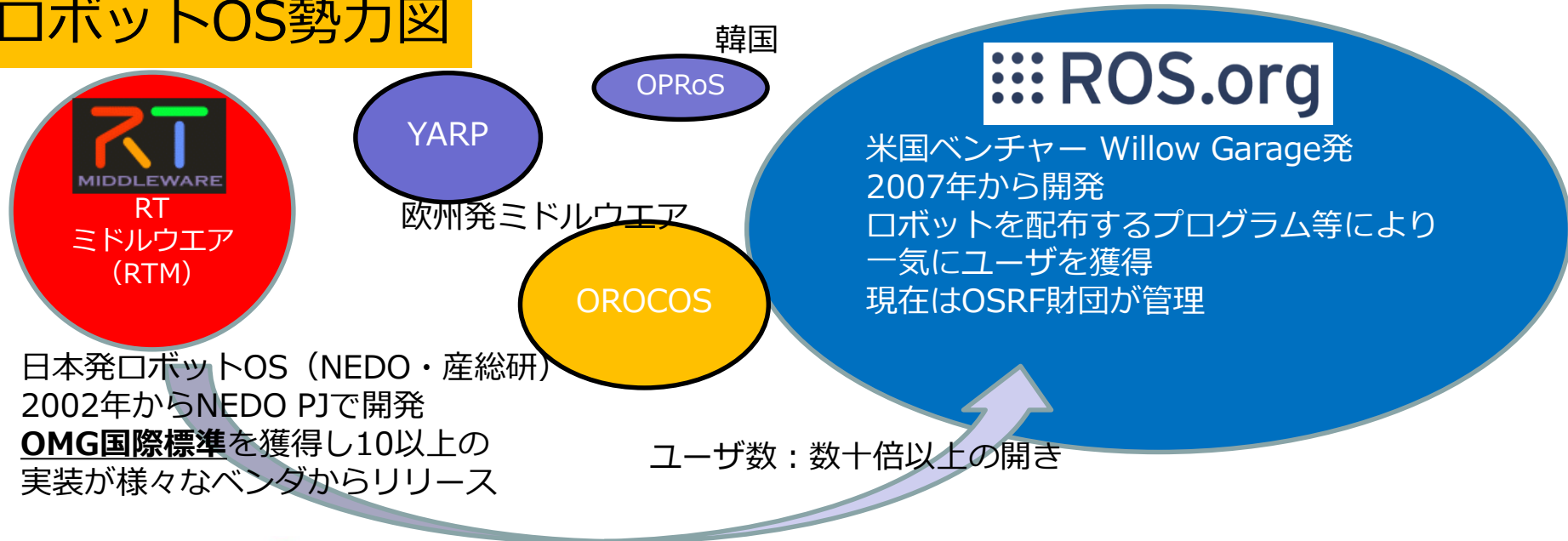
第3部: RTシステム構築実習

- 担当: 高橋三郎 氏 (産総研) 他
- 内容: OpenRTM-aistを利用し、人工知能技術を応用したロボットシステムを構築します。深層学習による画像認識を利用した移動ロボット制御システムを作成することで、実際の研究、開発へのアプリケーション応用について学びます。

ROSとの比較、動向

ロボットOS：現状と課題

ロボットOS勢力図



日本発ロボットOS (NEDO・産総研)
2002年からNEDO PJで開発
OMG国際標準を獲得し10以上の
実装が様々なベンダからリリース

課題



HONDA



- ROSはデファクト標準であり、OSRFにより仕様を変えられてしまう恐れあり。
 - ⇒ バージョンが上がるごとに、ユーザが振り回される
 - ⇒ ROSは技術的に古く、次期バージョンアップで大幅に変わる可能性あり。→ROS2へ
- 国内ユーザと海外ユーザの利用OSの差 (国内：Windows, ITRON, 海外：Linux)
 - ⇒ Linuxのみ対応するROSと、様々なOSに対応するRTMの利用比率は半々程度
- 国内でのソフトウェアの再利用が進んでいない。
 - ⇒ オープンソース文化の欠如。
 - ⇒ NEDO知能化PJで蓄積したソフトウェア資産も再利用が進んでいない。

ROSとRTM

ROSの特長

- UNIX文化に根差した設計思想
 - Windows等Linux以外は原則サポートしない
 - ROS2ではLinux以外もサポート
- 独自のパッケージ管理システムを持つ
- ノード（コンポーネント）の質、量ともに豊富
 - OSRFが直接品質を管理しているノードが多数
- ユーザ数が多い
- メーリングリストなどの議論がオープンで活発
 - コアライブラリの仕様が固定しにくい
- 英語のドキュメントが豊富

OpenRTMの特長

- 対応OS・言語の種類が多い
 - Windows、UNIX、uITRON、T-Kernel、VxWorks、QNX
 - Windowsでネイティブ動作する
- 日本国内がメイン
 - 日本語ドキュメント・ML・講習会
 - ユーザ数が少ない
- GUIツールがあるので初心者向き
 - コマンドラインツールの種類は少ない
 - rtshell
- 仕様が標準化されている
 - OMGに参加すればだれでも変更可
 - サードパーティー実装が作りやすい
 - すでに10程度の実装あり
- コンポーネントモデルが明確
 - オブジェクト指向、UML・SysMLとの相性が良い
 - モデルベース開発
- IEC61508機能安全認証取得
 - RTMSafety

ROS1→ROS2へ

OSRF内部の人のコメント

- NASAの仕事を請け負った時に、独自形式のROS messageはNASAでは使えないから、プロトタイプをROSで実装後にすべて作り直した
- NASAでは何らかの標準に準拠したものでないと使えない。その時は結局DDSを使用した。
- それ以外にもROS1では、1ノード1プロセス、コンポーネントモデルがないので、モデルベース開発にならない、ROS masterがSPOFになっているなど不都合な点が多々ある
- それ故、ROS2ではこれらの問題点を克服するため全く新しい実装にする予定。

ROS2

- 最新版：2016年12月 Beta版リリース
- 標準ミドルウェアの利用
 - 自前主義からの脱却（NASAや商用システムでは何らかの標準標準が求められるため）
 - 通信部分はOMG標準のDDS（Data Distribution Service、OMG規格）を採用
- コンポーネントモデルを導入することにした
 - RTMのように組み込み、性能を意識したアーキテクチャへ
- 対応OSの拡大
 - これまでは、ある特定のLinux（Ubuntu Linux）のみ
 - ROS2では、Windows、Macにも対応
 - ただし、リアルタイムOS（VxWorks、QNX等）への対応はなし

OMGによる通信規格DDSを採用



航空・軍事・医療・鉄道などで実績のある通信ミドルウェア標準

http://design.ros2.org/articles/ros_middleware_interface.html

ロボット新戦略(2015年2月)

- 日本経済再生本部決定文書
 - － ロボット革命実現会議の取りまとめ

pp.27-32

第2部 アクションプランー 五カ年計画

第1章 分野横断的事項

第3節 ロボット国際標準化への対応

第1項 ハードウェア/ソフトウェアのモジュール化を見据えた共通基盤 (ミドルウェア・ロボット OS)

(中略) ロボットを構成するハードウェア、ソフトウェアの機能要素をモジュール化・共通化することで、(中略) ロボットを低価格で構成することが可能となる。また、共通のソフトウェアプラットフォームを利用することで、(中略) ロボットインテグレータは対象毎の機能の実現に注力することができる。

(3) 2020 年に目指すべき姿

ロボットによる産業革命を日本主導で実現するためにも、(中略) ロボット制御の基幹部分であるロボット OS・ミドルウェアについても積極的な取組を進めていくことが重要である。(中略) 様々な国際標準を日本主導で策定することで、(中略) 日本のロボット産業が世界をリードすることが可能となる。

(中略) そのため、新たに設置される「ロボット革命イニシアティブ協議会」(第1項参照) が中心となって、必要な情報収集及び国際発信を行い、国際標準化の流れを主導していく。

ロボット新戦略

Japan's Robot Strategy

ービジョン・戦略・アクションプランー

日本経済再生本部
2015年2月10日

ロボット新戦略の重要性

関係各省庁も議論に参加し、内容の詳細に関与し合意したロボット戦略

→今後の施策に大きな影響

NEDO プラットフォームロボットPJ (H29年度～) (ロボット活用型市場化適用技術開発PJ)

ロボット革命イニシアティブ協議会 (RRI) での議論の結果を提言

ロボット活用型市場化適用技術開発プロジェクト

平成29年度予算案額 **17.5億円 (15.0億円)**

製造産業局 ロボット政策室
03-3501-1049

事業の内容

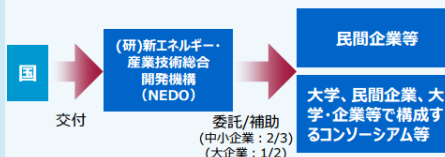
事業目的・概要

- 「ロボット新戦略」(平成27年2月10日 日本経済再生本部決定)に基づき、ものづくり分野とサービス分野において、これまで実現が困難であった組立工程における柔軟物把持等が可能となるロボットの技術開発を実施します。
- 加えて、小型汎用ロボットの導入コストの2割削減に向け、小型汎用ロボットの本体価格を引き下げ、汎用的な作業・工程に使える小型汎用ロボット(プラットフォームロボット)の開発(ハードウェア・ソフトウェアの共通化)を行います。

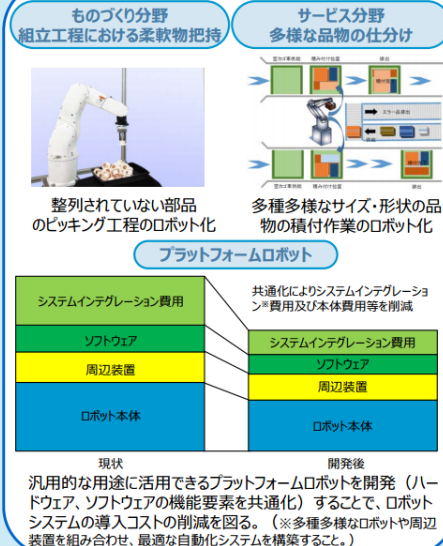
成果目標

- 平成27年度から平成31年度までの5年間の事業であり、平成31年度までに、上市可能なレベルのロボットを25種類以上開発することを目指す。

条件(対象者、対象行為、補助率等)



事業イメージ



「ロボット活用型市場化適用技術開発プロジェクト」に係る公募要領
【委託事業】

資料3

NEDO委託事業として 5月1日に公募開始

平成29年5月

国立研究開発法人新エネルギー・産業技術総合開発機構

ロボット・AI部

1

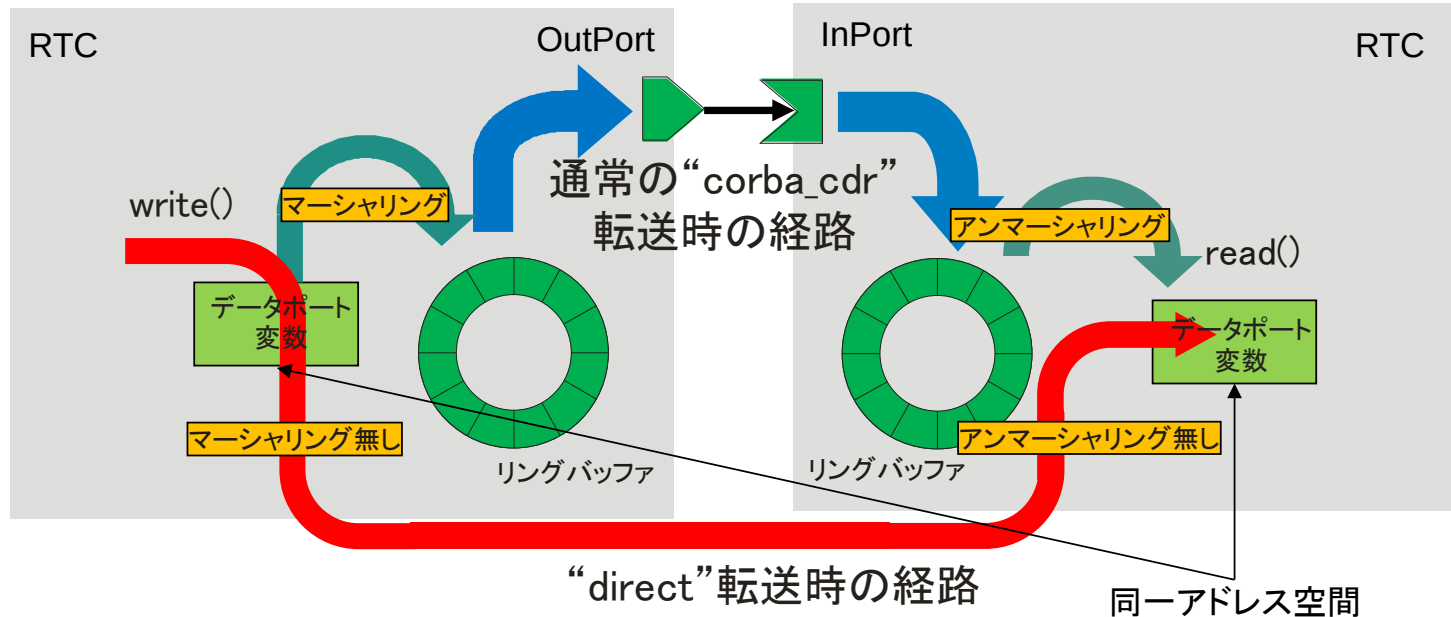
基本計画に明記

「NEDO事業で開発され標準となっている**RTミドルウェア**(OMG formal/08-04-01)又はデファクト標準となっている**ROS**(Robot Operating System)などの**利用を推奨**する。」

OpenRTM-aist-1.2の新機能 と今後の開発ロードマップ

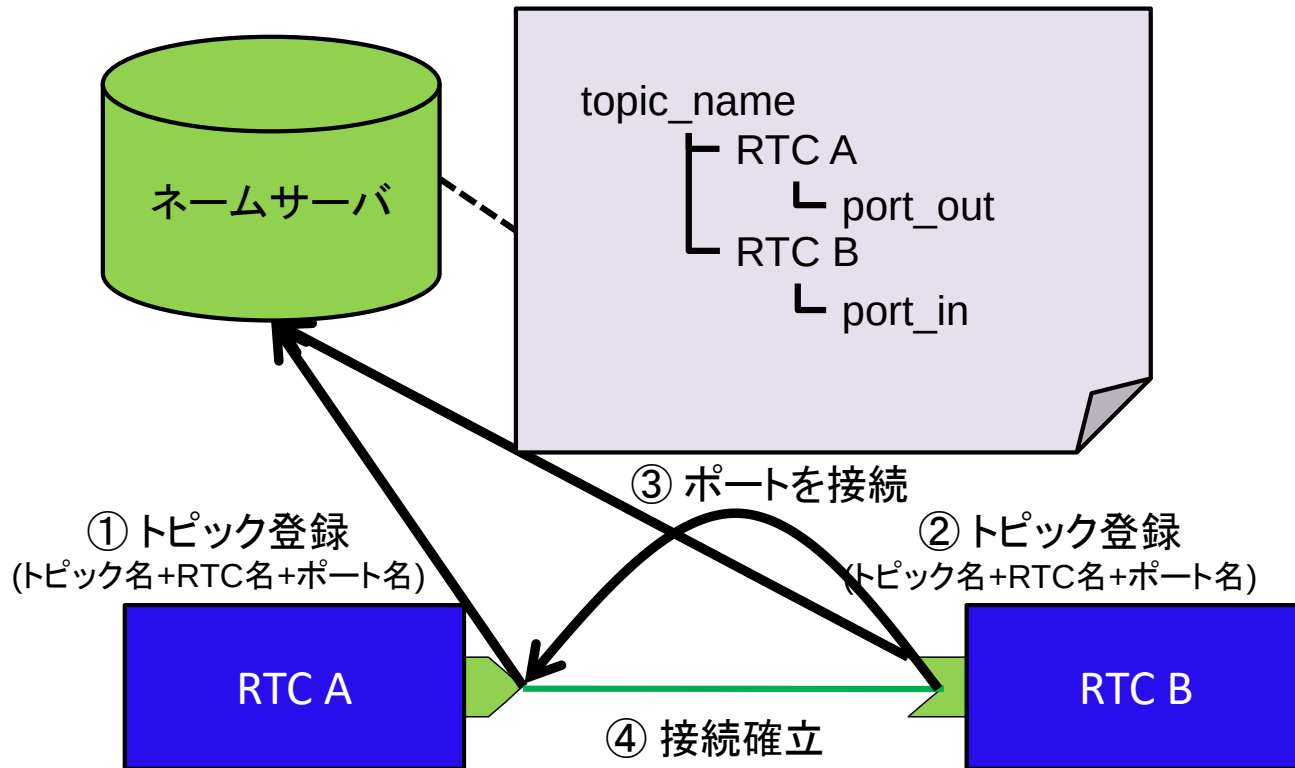
ダイレクト接続

プロセス境界



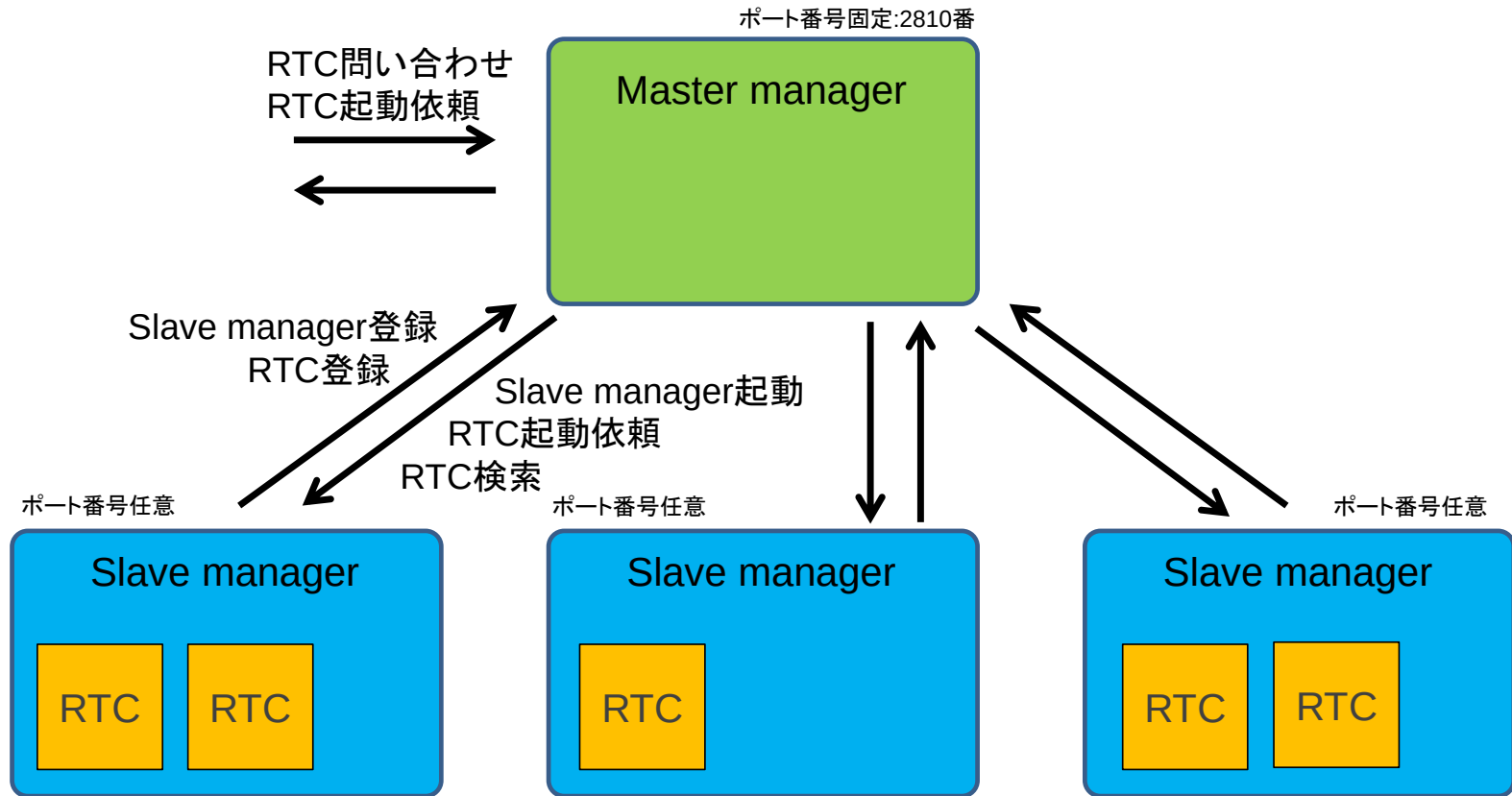
Direct接続：バッファ、マーシャリングをバイパスして直接変数領域でデータを受け渡し（変数間コピーなので高速）

トピック接続



トピック接続：相手のポートを特定せずに、同一のトピック（ラベル）を持つポート間で自動接続を行う機能。

マネージャ(rtcd)機能の拡張



コンポーネントの起動～終了（ライフサイクル）をリモートからすべて制御可能に。多数のノードを統一的に管理可能となり、運用時の効率向上が図れる。

その他の変更

- Windows
 - VS2017対応
 - VCバージョンチェンジャーツール提供
 - 32bit、64bit同時インストール可能
 - 同梱OpenCV 3.2へ変更
 - 同梱JRE OpenJDK8へ変更
 - Cmakeによるmsiパッケージ作成が可能
- Linux
 - OpenRTM-aist-Java, OpenRTPパッケージ提供
 - インストールスクリプト1本化 (C++, Python, Java, ツール)
 - Cmakeによる deb/rpmパッケージ作成が可能
 - Ubuntu (12.04, 14.04, 16.04, 16.10, 17.04)
 - Debian (7.0, 8.0)
 - Fedora (23,24,25)

2.0に向けて

- 新標準FSM4RTCの正式サポート
 - 状態遷移型RTCの枠組み
 - 5月12日 2A2J10 (RT ミドルウェアとオープンシステム) にて発表
- ROSやその他のミドルウェアとの連携機能
 - 要望多数
 - データポート相互接続
 - 運用時互換性等
- 開発効率向上のための機能
 - パッケージ化
 - 自動ビルド・CI
- 運用効率向上ための機能
 - ロギング機能、構成管理機能
 - RTC合成、組み込み機能
- サードパーティーRTC・ツール取り込み
- ROSノードの取り込み



Version 2.0
2017年度末

Version 2.1
2018年度中

Version 2.2
2018～2019年度